

Matricola: _____

Cognome: _____

Nome: _____

Basi di Dati - Laboratorio

III Appello del 16 Settembre 2026

Avvertenze: è severamente vietato consultare libri e appunti.

Durata 1h30min

DOMANDE

a) [3] Si consideri la seguente dichiarazione in PostgreSQL:

```
CREATE TABLE Auto (  
  targa CHAR(7),  
  num_telaio CHAR(17),  
  modello VARCHAR(32),  
  colore VARCHAR(32),  
  km INTEGER,  
  cilindrata INTEGER,  
  posti INTEGER,  
  proprietario CHAR(16)  
);
```

```
CREATE TABLE Prenotazione (  
  codice CHAR(16),  
  parcheggio CHAR(32),  
  posto CHAR(18),  
  auto CHAR(7),  
  data_ora_prenotazione TIMESTAMP,  
  data_ora_inizio_parcheggio TIMESTAMP,  
  data_ora_fine_parcheggio TIMESTAMP,  
  tipo CHAR(16),  
  pagato BOOLEAN,  
);
```

Su completi la dichiarazione (anche direttamente sopra su questo foglio) in modo che permetta di rappresentare:

- (i) il vincolo di chiave primaria sull'attributo **targa** di **Auto** e sull'attributo **codice** di **Prenotazione**,
- (ii) un vincolo di integrità referenziale da **Prenotazione.auto** a **Auto.targa**,
- (iii) una superchiave sull'attributo **num_telaio** di **Auto**,
- (iv) una superchiave composta dagli attributi **parcheggio**, **posto** e **data_ora_inizio_parcheggio** della tabella **Prenotazione**,
- (v) l'obbligatorietà dell'attributo **proprietario** nella tabella **Auto**, e degli attributi **tipo** e **data_ora_prenotazione** di **Prenotazione**,
- (vi) un vincolo di dominio per l'attributo **tipo** di **Prenotazione** che può assumere solo i seguenti valori: {'online-web', 'online-app', 'fisica'},
- (vii) un vincolo su **data_ora_prenotazione** < **data_ora_inizio_parcheggio**, e un vincolo su **data_ora_inizio_parcheggio** < **data_ora_fine_parcheggio**,
- (viii) un valore di default per **pagato** pari a FALSE.

ESERCIZI

b) Query SQL

Dato il seguente schema relazionale,

AUTO(targa, num_telaio, modello, colore, km, cilindrata, posti, proprietario)

PRENOTAZIONE(codice, parcheggio, posto, auto, data_prenotazione, data_ora_inizio, data_ora_fine, tipo, pagato)

PARCHEGGIO(nome, comune, indirizzo, coperto)

Chiavi primarie sottolineate e vincoli di integrità:

PRENOTAZIONE.auto → AUTO,

PRENOTAZIONE.parcheggio → PARCHEGGIO,

Formulare in SQL le seguenti interrogazioni:

i. [5] Trovare tutti i parcheggi in cui vengono effettuate più prenotazioni online (tipo = 'online-web' oppure tipo = 'online-app') che prenotazioni fisiche. Per ciascun parcheggio riportare il nome, il comune e l'indirizzo in cui si trova.

```
SELECT r.parcheggio
FROM prenotazione r
WHERE r.tipo <> 'fisica'
GROUP BY r.parcheggio
HAVING COUNT(*) > ( SELECT count(*)
                     FROM prenotazione r1
                     WHERE r1.categoria = 'fisica' AND
                           r1.parcheggio = r.parcheggio )
```

ii. [4] Trovare tutti i parcheggi in cui sono mai state fatte prenotazioni di tipo 'fisica' nel 2024. Per ciascun parcheggio riportare il nome ed il numero delle prenotazioni complessive.

```
SELECT nome, COUNT(*)
FROM parcheggio p JOIN prenotazione r ON ( p.nome = r.parcheggio )
WHERE p.nome NOT IN ( SELECT r1.parcheggio
                      FROM prenotazione r1
                      WHERE r1.data_ora_prenotazione > '01/01/2024' and
                            r1.data_ora_prenotazione < '01/01/2024' and
                            r1.tipo = 'fisica' )

GROUP BY p.nome;
```

iii. [5] Trovare quanti parcheggi del comune di 'Verona' hanno avuto un numero di prenotazioni complessive nel 2024 superiore al numero medio di prenotazioni complessive del 2024 dei parcheggi del comune di 'Padova'.

```
CREATE VIEW nprenotazioni AS
( SELECT r.codice, r.comune, COUNT(*) as num
  FROM prenotazione r join parcheggio p on (r.parcheggio = p.codice)
  WHERE data_prenotazione > '01/01/2024' and data_prenotazione < '01/01/2024'
  GROUP BY r.codice, r.comune
)
```

```
SELECT count(*)
FROM nprenotazioni n
WHERE n.comune = 'Verona'
AND n.num > ( SELECT AVG( n1.num )
              FROM nprenotazioni n1
              WHERE n1.comune = 'Padova' );
```

iv. [3] Trovare tutti i parcheggi coperti del comune di Verona. Indicare che tipo di indici è possibile costruire per ottimizzare l'interrogazione, giustificando la risposta.

```
SELECT *  
FROM parcheggio  
WHERE comune = 'Verona' AND coperto = true;
```

c) **Query MongoDB**

[3] Si supponga che le informazioni relative alla tabella PARCHEGGIO dell'esercizio precedente siano memorizzate in un documento Json con la seguente struttura:

```
parcheggio{ nome: '...', comune: '...', indirizzo: '...', coperto: ... }
```

Formulare una query in MongoDB per trovare tutti i parcheggi del comune di Verona che non sono coperti.

g) **Transazioni**

[6] Descrivere la differenza tra l'anomalia di aggiornamento fantasma e l'anomalia di lettura inconsistente anche attraverso l'utilizzo di esempi di transazioni SQL che possono produrre tale anomalia. Indicare il tipo di isolamento minimo che deve essere utilizzato per evitare ciascun tipo di anomalia.

h) **Python**

[4] Si consideri una base di dati in PostgreSQL contenente la seguente tabella

```
PRENOTAZIONE( codice, parcheggio, posto, auto, data_prenotazione, data_ora_inizio, data_ora_fine, tipo, pagato )
```

scrivere il codice di un programma Python che richieda all'utente di inserire il codice del parcheggio ed una data, e restituisca l'elenco dei posti auto già prenotati per quella data ($\text{data_inizio_parcheggio} \leq \text{data}$ e $\text{data_fine_soggiorno} \leq \text{data}$). L'elenco riporterà le prenotazioni ordinate per data di inizio. Se non esistono prenotazioni attive nella data indicata per il parcheggio selezionato, verrà riportato il messaggio "Nessuna prenotazione attiva per il parcheggio X nella data Y", dove X e Y vanno sostituiti con il codice del parcheggio e la data inserite dall'utente.